기말 프로젝트 과제 명세서

# LLM 에이전트 기반 자동 댓글 답장 서비스 개발

FastAPI · Streamlit · LangChain Agent · Webhook

## 프로젝트 목표

외부 플랫폼에서 발생한 댓글을 웹훅으로 수신하고, LLM 에이전트가 댓글의 의도와 맥락을 분석하여 적절한 답장 초안을 생성하는 풀스택 AI 서비스를 구현한다. FastAPI 백엔드, LangChain 에이전트, Streamlit 관리 대시보드를 하나의 시스템으로 통합한다.

| 항목 | 내용 |
| --- | --- |
| 개발 형태 | 개인 또는 팀 프로젝트(수업 운영 기준에 따라 결정) |
| 필수 기술 | Python, FastAPI, Pydantic, LangChain, ChatOpenAI, Streamlit, requests |
| 데이터 저장 | 메모리 또는 SQLite 권장 |
| 최종 제출 | 소스코드, requirements.txt, README.md, 실행 화면 및 테스트 결과 |

※ 실제 외부 플랫폼 연동이 어려운 경우, 테스트용 웹훅 요청을 직접 전송하는 방식으로 필수 기능을 검증한다.

# 1. 프로젝트 배경 및 개발 목적

온라인 커뮤니티, 쇼핑몰, 학교 서비스 및 기업 SNS에는 반복적으로 유사한 질문과 의견이 등록된다. 운영자가 모든 댓글을 직접 확인하고 답변하면 시간이 많이 들며, 답변 품질과 응답 속도가 일정하지 않을 수 있다. 본 프로젝트에서는 댓글 수신부터 답변 생성, 검토, 이력 관리까지의 흐름을 자동화하는 AI 기반 댓글 관리 서비스를 개발한다.

## 1.1 핵심 학습 목표

- 웹훅의 요청·응답 구조와 외부 시스템 이벤트 수신 방식을 이해하고 구현한다.
- Pydantic 모델을 이용해 입력 데이터의 형식과 필수 항목을 검증한다.
- FastAPI에서 비동기 작업, 예외 처리, REST API 설계를 적용한다.
- LangChain 에이전트와 커스텀 툴을 연결해 단순 생성형 답변을 넘어 근거 기반 답변을 설계한다.
- Streamlit 대시보드에서 API를 호출하고 댓글·답장 이력과 시스템 설정을 관리한다.
- 테스트, 로그, 보안, 오류 대응을 고려해 실행 가능한 서비스 형태로 통합한다.

# 2. 구현할 시스템의 전체 구조

다음 흐름을 기준으로 모듈을 설계한다. 각 모듈은 역할을 분리하되, 하나의 실행 가능한 애플리케이션으로 연결되어야 한다.

| 단계 | 구성 요소 | 주요 역할 |
|---|---|---|
| ① 댓글 수신 | 외부 플랫폼 또는 테스트 클라이언트 | 댓글 이벤트를 POST /webhook/comment로 전송 |
| ② 검증·기록 | FastAPI + Pydantic + 저장소 | 입력 검증, 중복 확인, 댓글 이력 저장, 처리 상태 등록 |
| ③ 비동기 처리 | BackgroundTasks 또는 동등한 방식 | 웹훅 응답을 먼저 반환하고 답변 생성을 별도로 수행 |
| ④ 판단·생성 | LangChain Agent + ChatOpenAI | 댓글 의도 파악, 필요 시 툴 호출, 답장 초안 생성 |
| ⑤ 결과 저장 | SQLite 또는 메모리 저장소 | 생성 답변, 처리 상태, 오류 내용 및 시간 저장 |
| ⑥ 관리·조회 | Streamlit + FastAPI REST API | 댓글 목록, 답장, 상태, 설정 조회 및 제어 |

## 2.1 권장 데이터 흐름

댓글 수신 → 데이터 검증 → 댓글 이력 저장(상태: 대기) → 웹훅에 빠른 응답 반환 → 에이전트 작업 실행 → 커스텀 툴 조회(필요한 경우) → 답장 생성 → 결과 저장(상태: 완료 또는 실패) → 대시보드에서 결과 확인 순서로 구현한다.

## 2.2 필수 화면

- 대시보드: 전체 댓글 수, 대기·완료·실패 건수, 최근 댓글 목록을 표시한다.
- 댓글 상세: 작성자, 댓글 내용, 게시글 ID, 수신 시각, 생성 답장, 처리 상태를 확인한다.
- 설정: 자동 답장 활성화 여부, 답변 어조 또는 기본 지침을 변경한다.
- 오류 확인: 처리 실패 사유를 확인하고 재처리하거나 상태를 갱신할 수 있도록 설계한다.

# 3. 분야별 상세 요구사항

## 3.1 분야 1 — 백엔드: FastAPI

### A. 웹훅 엔드포인트

- POST /webhook/comment를 구현한다. 댓글 이벤트를 JSON 형식으로 수신해야 한다.
- Pydantic 모델로 최소한 comment_id, post_id, author, content를 검증한다. 플랫폼명과 created_at은 선택 항목으로 둘 수 있다.
- 필수 값 누락, 자료형 오류, 공백 댓글 등 잘못된 입력에 적절한 4xx 응답을 반환한다.

- 정상 요청은 댓글을 기록한 뒤 처리 접수 결과를 반환한다. LLM 생성이 끝날 때까지 요청을 불필요하게 기다리지 않는다.

### B. 처리 상태 및 비동기 실행

- 댓글 상태를 최소한 pending(대기), processing(처리 중), completed(완료), failed(실패)로 구분한다.
- FastAPI BackgroundTasks 또는 이에 준하는 비동기 실행 방식을 사용한다. 작업 시작·완료·실패 상태가 저장되어야 한다.
- BackgroundTasks는 교육용·소규모 구현에 적합하다. 서버 재시작에도 작업이 보존되어야 하는 확장형 설계는 별도 작업 큐를 고려할 수 있다.

### C. 데이터 관리 및 조회 API

- 댓글 원문, 생성된 답장, 처리 상태, 수신·처리 시각, 오류 메시지를 저장한다. 메모리 저장은 서버 재시작 시 사라진다는 점을 README에 명시한다.
- GET /api/comments: 댓글 목록과 생성 답장 이력을 반환한다.
- GET /api/comments/{comment_id}: 특정 댓글의 상세 정보를 조회한다.
- GET /api/settings 및 PATCH /api/settings: 설정 조회·변경 API를 제공한다.
- 선택 기능: POST /api/comments/{comment_id}/retry로 실패한 작업을 재시도한다.

### D. 오류 처리·안정성

- LLM API 오류, 타임아웃, 툴 실행 오류, 저장 오류를 구분해 기록한다.
- 외부 API 오류가 발생해도 서버 전체가 중단되지 않도록 예외를 처리한다.
- 동일 comment_id가 재전송될 경우 중복 댓글이나 중복 답장이 생성되지 않도록 중복 처리 방식을 구현한다.
- API 키는 코드에 직접 기록하지 않고 환경변수(.env 등)로 관리한다. .env 파일은 저장소에 제출하지 않는다.

## 3.2 분야 2 — 프론트엔드: Streamlit

### A. 대시보드

- Streamlit에서 FastAPI의 API를 requests 등으로 호출한다.
- 최근 댓글 목록을 표 또는 카드 형태로 표시하고, 각 댓글의 작성자·내용·수신 시각·처리 상태·AI 답장을 확인할 수 있어야 한다.
- 새로고침 버튼 또는 자동 갱신 기능을 제공한다. 자동 갱신을 구현하지 않더라도 사용자가 최신 이력을 조회할 수 있어야 한다.

### B. 설정·제어

- 자동 답장 기능의 활성화/비활성화 토글을 제공한다.
- 기본 답변 어조(예: 친절하고 전문적), 답변 길이 또는 추가 지침 중 최소 1개 이상을 설정할 수 있도록 한다.
- 자동 답장이 비활성화된 경우 댓글은 저장하되 답변 생성은 수행하지 않거나, 초안만 생성하는 등 정책을 명확히 정한다.

### C. 사용성 및 오류 표시

- 백엔드 서버가 실행되지 않았거나 API 요청이 실패한 경우 사용자가 이해할 수 있는 메시지를 표시한다.
- 댓글이 없는 경우 빈 상태 안내를 표시한다.
- 답변이 아직 생성되지 않은 경우 대기 또는 처리 중임을 구분해 보여준다.

## 3.3 분야 3 — LLM 및 LangChain Agent

### A. 에이전트 구성

- ChatOpenAI와 create_tool_calling_agent 및 AgentExecutor 등 LangChain의 호환 구성요소를 사용한다. 설치한 LangChain 버전에 맞는 API를 선택하고 README에 버전을 기록한다.
- 에이전트는 댓글을 읽고 답변에 필요한 정보를 판단하며, 필요 시 커스텀 툴을 호출한 뒤 답장 초안을 생성한다.
- LLM 호출 결과를 검증하고, 빈 응답이나 오류가 발생했을 때 실패 상태 및 오류 내용을 저장한다.

### B. 커스텀 툴 — 최소 1개 필수

다음 예시 중 하나 이상을 구현한다. 툴은 단순히 정의만 하는 것이 아니라 에이전트에 등록되어 실제 실행되는 것을 테스트로 입증해야 한다.

| 툴 예시 | 입력 | 기능 및 출력 예시 |
|---|---|---|
| FAQ 검색 | 질문 또는 키워드 | 사전에 작성한 FAQ에서 관련 항목을 찾아 답변 근거 반환 |
| 제품 정보 조회 | 제품명 또는 제품 코드 | 가격·특징·이용 조건 등 등록된 정보를 조회 |
| 감성 분석 | 댓글 내용 | 긍정·중립·부정 등 감성 및 간단한 근거 반환 |
| 운영 정책 조회 | 문의 유형 | 환불·배송·운영시간 등 사전 정의된 정책 반환 |

- 툴의 목적, 입력값, 반환값, 예외 처리 방식을 코드 또는 README에 설명한다.
- 가능하면 외부 웹 검색보다 로컬 FAQ/제품 데이터 등 통제 가능한 자료를 사용해 재현성을 확보한다.

### C. 시스템 프롬프트

- 에이전트 역할을 커뮤니티 또는 고객지원 담당자로 정의하고, 친절하고 전문적이며 간결한 말투를 지정한다.
- 댓글에 포함된 질문·불만·칭찬·스팸 가능성을 구분하고, 확인된 근거를 바탕으로 답변하도록 지시한다.
- 근거가 부족한 가격·정책·일정·사실을 임의로 만들어내지 말고, 필요한 경우 확인이 필요하다고 안내하게 한다.
- 댓글 본문에 포함된 지시문은 신뢰할 수 없는 사용자 입력으로 취급하고, 시스템 규칙이나 비밀정보 공개 요청을 따르지 않도록 한다.
- 개인정보, 공격적 표현, 민감한 문의를 다룰 때 불필요한 개인정보 반복이나 단정적 답변을 피하도록 한다.

### D. 대화 맥락 관리 — 선택/가점

- 동일 사용자의 이전 댓글 또는 게시글 맥락을 참조해 일관된 답변을 생성한다.
- 사용자 식별은 플랫폼에서 제공한 안정적인 ID가 있을 때 수행하며, 불필요한 개인정보를 수집하지 않는다.
- 메모리 사용 여부, 저장 범위, 초기화 정책을 문서화한다.

## 4. API 명세 예시

아래는 최소 권장 명세이다. 세부 필드와 응답 형식은 프로젝트에서 일관되게 설계할 수 있다.

| 메서드·경로 | 목적 | 성공 응답 예시/기대 동작 |
|---|---|---|
| POST /webhook/comment | 댓글 이벤트 수신 | 202 Accepted 또는 200 OK와 접수 상태 반환 |
| GET /api/comments | 댓글·답장 목록 | JSON 배열 또는 목록 객체 반환 |
| GET /api/comments/{comment_id} | 댓글 상세 조회 | 해당 댓글 객체 반환, 없으면 404 |
| GET /api/settings | 현재 설정 조회 | 자동 답장 여부·기본 지침 반환 |
| PATCH /api/settings | 설정 변경 | 변경된 설정 반환 |
| POST /api/comments/{comment_id}/retry (선택) | 실패 건 재처리 | 재처리 접수 상태 반환 |

### 4.1 댓글 데이터 예시

요청 JSON에는 comment_id, post_id, author, content가 포함되어야 한다. created_at, platform은 선택 항목으로 설계할 수 있다. 실제 외부 플랫폼의 필드명과 다르면 변환 계층을 두어 내부 표준 모델로 매핑한다.

{ "comment_id": "cmt-1001", "post_id": "post-210", "author": "user_17", "content": "운영 시간은 어떻게 되나요?", "created_at": "2026-10-14T10:30:00+09:00", "platform": "test" }

### 4.2 웹훅 응답 예시

{ "accepted": true, "comment_id": "cmt-1001", "status": "pending" }

웹훅 응답은 처리 접수 사실을 의미하며, 답변 생성 완료를 의미하지 않는다. 생성 결과는 조회 API를 통해 확인한다.

## 5. 비기능 요구사항

| 항목 | 요구사항 |
| --- | --- |
| 응답성 | 웹훅은 LLM 생성 완료를 기다리지 않고 신속히 접수 응답을 반환한다. |
| 신뢰성 | 실패 상태와 원인을 기록하고, 동일 이벤트 중복 처리를 방지한다. |
| 보안 | API 키는 환경변수로 관리하고, 로그에 비밀키나 불필요한 개인정보를 남기지 않는다. |
| 유지보수성 | 백엔드, 에이전트/툴, 저장소, 프론트엔드의 책임을 함수 또는 모듈로 분리한다. |
| 재현성 | requirements.txt와 실행 절차를 제공하며, 테스트 입력 예시를 포함한다. |
| 사용성 | 대시보드에서 댓글·답장·상태·설정을 쉽게 확인하고 변경할 수 있다. |

## 6. 테스트 및 검증 기준

다음 테스트를 수행하고 결과를 README 또는 별도 테스트 결과 문서에 기록한다.

- 정상 댓글을 웹훅으로 전송하면 저장되고, 접수 응답을 받는다.
- 필수 필드 누락 또는 잘못된 데이터 형식에 대해 4xx 응답이 반환된다.
- 댓글 수신 직후에는 pending/processing, 처리 완료 후에는 completed 상태가 조회된다.
- 커스텀 툴이 에이전트에 등록되어 실제 호출되고, 결과가 답변 생성에 활용된다.
- LLM API 키 오류 또는 호출 실패가 발생하면 failed 상태와 오류 요약이 기록된다.
- 동일 comment_id를 두 번 전송했을 때 중복 처리 정책이 적용된다.
- Streamlit에서 댓글 목록과 답장 이력을 조회할 수 있다.
- 대시보드의 설정 변경이 FastAPI 설정 API와 연결되고 실제 동작에 반영된다.
- 자동 답장 비활성화 시 정의한 정책대로 생성이 중단되거나 초안 모드로 처리된다.

## 7. 제출물 및 프로젝트 실행

### 7.1 필수 제출물

- 전체 소스코드: FastAPI 백엔드, LangChain 에이전트/툴, Streamlit 프론트엔드
- requirements.txt: 실행에 필요한 라이브러리와 가능한 경우 버전 정보
- README.md: 개발 환경, 환경변수 설정, 설치 및 실행 방법, API 설명, 툴 설명, 테스트 방법
- 실행 증빙: 대시보드 화면 캡처와 정상·오류 테스트 결과
- 환경 설정 예시: .env.example 등 키 값을 제외한 설정 템플릿

### 7.2 권장 프로젝트 구조

comment-reply-agent/ ├── backend/ │ ├── main.py # FastAPI 앱 및 라우트 │ ├── schemas.py # Pydantic 모델 │ ├── storage.py # 데이터 저장·조회 │ ├── agent.py # LangChain 에이전트 │ ├── tools.py # 커스텀 툴 │ └── settings.py # 시스템 설정 ├── frontend/ │ └── app.py # Streamlit 대시보드 ├── tests/ │ └── test_webhook.py ├── requirements.txt ├── .env.example ├── .gitignore └── README.md

### 7.3 실행 방법 문서에 포함할 항목

- 가상환경 생성 및 의존성 설치 명령
- OPENAI_API_KEY 등 필수 환경변수 설정 방법
- FastAPI 실행 명령과 Streamlit 실행 명령

- 테스트 댓글을 전송하는 curl 또는 Python 예제
- 외부 플랫폼 없이 테스트하는 방법과 알려진 제한사항

## 8. 평가 기준

| 평가 영역 | 배점 | 평가 관점 |
|---|---|---|
| FastAPI 웹훅·데이터 정합성 | 25점 | 입력 검증, 상태 관리, 조회 API, 중복 처리, 오류 대응 |
| LangChain 에이전트·툴 활용 | 30점 | 에이전트 구성, 커스텀 툴 실제 호출, 프롬프트의 적절성, 답변 근거 |
| Streamlit UI·API 연동 | 20점 | 목록·상세·상태 표시, 설정 토글, API 오류 표시 및 사용성 |
| 통합 테스트·안정성 | 15점 | 정상/오류 테스트, 비동기 처리, 로그 및 보안 고려 |
| 제출 문서·실행 재현성 | 10점 | 소스 정리, requirements.txt, README, 실행 증빙 |

총점: 100점. 대화 맥락 관리, 재시도 기능, SQLite 영속성, 테스트 자동화, 응답 품질 평가 등은 추가 가점 항목으로 운영할 수 있다.

## 9. 개발 단계 및 권장 진행 순서

| 단계 | 개발 작업 | 완료 기준 |
|---|---|---|
| 1단계 | Pydantic 모델과 웹훅 구현 | 정상·오류 요청이 검증되고 저장됨 |
| 2단계 | 저장소와 조회 API 구현 | 댓글 목록 및 상세 조회 가능 |
| 3단계 | LangChain 에이전트·툴 구현 | 툴 호출을 포함한 답장 생성 확인 |
| 4단계 | 비동기 처리 및 상태 관리 | 웹훅 빠른 응답, 완료·실패 상태 반영 |
| 5단계 | Streamlit 대시보드 구현 | 목록·답장·설정이 API와 연동됨 |
| 6단계 | 통합 테스트 및 문서화 | README에 따라 다른 환경에서도 실행 가능 |

## 10. 유의사항 및 설계 원칙

- 실제 플랫폼의 자동 댓글 게시 API까지 반드시 연결할 필요는 없다. 본 과제의 필수 범위는 댓글 수신, 답장 생성, 결과 관리이며, 실제 게시 기능은 플랫폼 정책과 인증 조건에 따라 선택 구현한다.
- 자동 답장 토글이 활성화되어도 처음에는 'AI 답장 초안 생성'을 기본으로 하고, 실제 게시 전 사람의 검토를 거치도록 설계하는 것을 권장한다.
- 웹훅 요청을 보낸 측에서 재전송할 수 있으므로 중복 이벤트 처리 기준을 정한다.
- 외부에서 접근 가능한 웹훅을 배포할 경우 서명 검증 또는 공유 비밀 토큰 등 인증 방식을 고려한다. 테스트 환경에서는 구현 범위와 한계를 문서화한다.
- LLM의 출력은 항상 정확하다고 가정하지 않는다. FAQ나 운영 정책 등 근거 자료를 툴로 제공하고, 불확실한 답변은 단정하지 않도록 한다.

---

본 명세서는 수업 과제 안내 및 평가에 사용할 수 있도록 필수 기능과 선택 기능을 구분한 기준 문서이다. 세부 구현 방식은 제시된 API 계약과 검증 기준을 충족하는 범위에서 선택할 수 있다.